

Course Notes for *Hashcat Essential Training*

This document contains links and notes for the course *Hashcat Essential Training* from LinkedIn Learning.

Challenges are provided in a separate file, [Challenges/Challenges.pdf](#). Solutions to these challenges are provided in [Challenges/Solutions.pdf](#).

Links

Platform-Specific Software

This table provides links to OpenCL toolkits and vendor software that allows hashcat to use GPUs and CPUs for its calculations.

	Windows	Linux
Intel Core / Xeon CPU (OpenCL)	Learn more	Learn more
Intel iGPU (OpenCL)	Learn more	Learn more
NVIDIA GPU (CUDA)	Learn more	Learn more
AMD GPU (ROCm)	Learn more	Learn more

You can learn more about pocl at the project's [website](#) and [GitHub repository](#).

Explore the Apple Metal framework on [Apple's Developer site](#).

Hashcat Links

[Hashcat website](#)

[Hashcat wiki](#)

[Hashcat FAQ](#)

[Hashcat forum](#)

[Hashcat on GitHub](#)

Other Useful Links

[Using a GPU with CUDA in Windows Subsystem for Linux](#)

[Install the Homebrew package manager on macOS](#)

[John the Ripper on GitHub](#)

[Hashcat example hashes](#)

[HashID on GitHub](#)

[SecLists on GitHub](#)

[Hashcat Wiki entry on Association Attacks](#)

[Hashcat Wiki entry on Rule-based Attacks](#)

[Hashcat Forum post introducing the Brain service](#)

[Hashtopolis on GitHub](#)

LinkedIn Learning Courses

[Linux: Multitasking at the Command Line](#)

[Learning Linux Command Line](#)

[Windows Terminal Skills](#)

Tips and notes

Generating a list of hashes in the shell

The following hashes provided with this course were generated from strings in the file `strings.txt`:

- `md5.hashes` (using `md5sum`)
- `sha1.hashes` (using `sha1sum`)
- `sha256.hashes` (using `sha256sum`)
- `sha512.hashes` (using `sha512sum`)
- `sha512-crypt.hashes` (using `openssl passwd -6`)

```
# Read from a file called strings.txt and generate a hash for each line. Output the
results into md5.hashes or similar.
# Replace md5sum with other tools like sha1sum, sha256sum, etc.
while read -r line; do echo -n $line | md5sum | tr -d " -" >> md5.hashes; done <
strings.txt

# Use openssl-passwd to generate SHA-512 crypt hashes
while read -r line; do openssl passwd -6 $line | tr -d " -" >> sha512-crypt.hashes;
done < strings.txt
```

Working with different encodings

Some strings may use different encodings. The `iconv` tool allows us to convert input from one encoding to another. Consult the manpages for `iconv` and `xxd` for more details.

```
# Convert a string from (-f) UTF-8 to (-t) ISO-8859-1 and then generate a hash of the
encoded string
echo -n "äpfel123" | iconv -f UTF8 -t ISO-8859-1 | md5sum
# -> b8e840de59ed6aa0b4d55adff416cdf3 -
```

```
# Convert a hex string to readable text
# b8e840de59ed6aa0b4d55adff416cdf3:$HEX[e47066656c313233]
echo "e47066656c313233" | xxd -p -r | iconv -f ISO-8859-1
# -> äpfel123
```

If a password contains special Unicode characters, we may need to use more than one mask token to accommodate them. Some characters are two bytes wide, and others, like certain emoji, are four bytes wide.

```
# Create a hash using UTF-8
echo -n "äpfel123" | md5sum
# -> a215d343095af5f754aab929ead1374e -
```

```
# Using the ?b token to generate hexadecimal byte values.
# Using two ?b tokens to accommodate characters with a width of two bytes, which ä is
in UTF-8 encoding (C3 A4).
hashcat -m 0 'a215d343095af5f754aab929ead1374e' -a 3 '?b?bpfel123' -0 -o results.txt
# -> a215d343095af5f754aab929ead1374e:äpfel123
```

Re-running attacks

Once a hash has been cracked, hashcat stores the resulting hash and password in the potfile, and it will skip that hash in future cracking attempts.

If you plan to run different attacks on the same hashlist to develop different cracking strategies, it may be useful to either delete the potfile on your system periodically, to reset hashcat's state, or to redirect the potfile to a temporary one with the `--potfile-path=` option instead.